

Design, Implementation, and Deployment of a Tool-Augmented Conversational Agent for a Personal Portfolio Website

Youssef Ramadan

Computer & Communication Engineering Program, Alexandria University

Abstract

This report documents the design and implementation of a tool-augmented conversational agent embedded in a personal portfolio website. The system exposes a small set of function-calling tools backed by a structured personal data file, allowing a hosted language model to answer visitor questions about the author's background, skills, and projects using retrieval against real data rather than a static FAQ or scripted responses. The initial implementation used a FastAPI backend calling the Google Gemini API through the google-genai Python SDK, with a lightweight vanilla JavaScript chat widget as the frontend. The report covers the system architecture, backend and frontend implementation, containerization, a detailed case study of the deployment and integration failures encountered and resolved during development, and an evaluation of free-tier hosting options for both API-based and self-hosted local-model (Ollama) deployment. The debugging case study is presented as an accurate account of issues actually encountered, since third-party API and hosting providers changed authentication formats, model availability, and free-tier policies during the development period.

1. Introduction

Personal portfolio websites conventionally present static text: a biography, a skills list, and project descriptions. This project explores replacing part of that static presentation with an interactive agent that a visitor can query directly, with responses grounded in the author's actual structured data rather than free-form generation alone. The motivating design constraint was that the agent's claims about the author's background should be traceable to a single source of truth (a JSON profile document) rather than left to the language model's unconstrained recall, which motivated a function-calling (tool-use) architecture over a simple system-prompt-only chatbot.

A secondary goal of the project, which occupies a substantial portion of this report, was practical: building and operating a small production service end-to-end, including authentication, deployment, containerization, and free-tier hosting economics, all of which surfaced real engineering problems independent of the AI component itself.

2. System Architecture

The system follows a three-tier architecture: a static frontend chat widget, a stateless backend API that mediates between the frontend and the language model provider, and a structured data file that the model can query through tool calls rather than through its own training data. Figure 1 summarizes the request flow.

```
+-----+
| Browser (index.html / |
| chat.js chat widget) |
+-----+
```

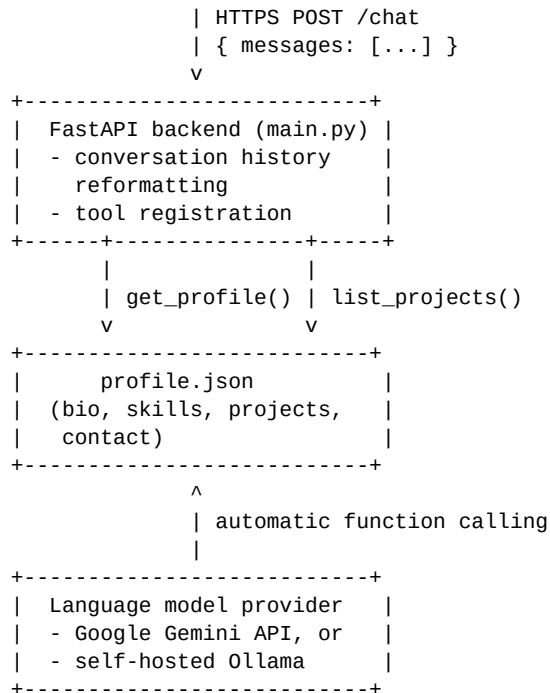


Figure 1. Request flow from the browser chat widget through the backend to the model provider and back.

This design keeps the frontend and the model provider fully decoupled: the browser only ever talks to the FastAPI backend, and swapping the model provider (as later occurred, moving from a hosted API to a self-hosted local model) requires no frontend changes.

3. Backend Implementation

3.1 API Design

The backend is a single-endpoint FastAPI application. A POST request to /chat accepts a JSON body containing the full conversation history as a list of role-tagged messages, validated with Pydantic models. On each request, the backend reconstructs the conversation as SDK-native history objects, opens a chat session against the model, sends the newest user message, and returns the model's reply as JSON. Cross-origin requests are permitted so that the static frontend, served independently of the backend, can reach the API.

3.2 Tool-Augmented Responses

Two plain Python functions, `get_profile` and `list_projects`, are registered with the model provider's automatic function-calling mechanism. Each function reads and returns data from a single JSON file rather than any hard-coded or model-recalled content. This constrains the agent's factual claims about the author to a single, auditable source document, and means updating the portfolio's content requires editing only the JSON file rather than any prompt or code.

3.3 Personal Data Schema

The profile document captures five top-level fields: identity and biography (name, tagline, location, bio), a flat skills list, a projects array (each with name, description, technology tags, and an optional repository

link), and a contact block (email, GitHub, LinkedIn). Table 1 summarizes the schema.

Field	Type	Purpose
name, tagline, bio	string	Identity and summary shown to the agent as ground truth
skills	array[string]	Flat list surfaced verbatim in responses
projects	array[object]	name, description, tech, link per project
contact	object	email, github, linkedin

Table 1. Structure of the profile.json data source.

4. Frontend Implementation

The frontend is intentionally dependency-free: a single HTML page, one CSS file, and one vanilla JavaScript file with no build step or framework. The chat widget maintains an in-memory message array, appends user and assistant messages to a scrolling log with timestamps, shows a typing indicator while awaiting the backend response, and degrades gracefully to an inline error message if the fetch request fails. This keeps the client trivially deployable as static files on any static host, independent of the backend's own hosting.

5. Containerization

The backend is packaged with a minimal Dockerfile based on python:3.12-slim: dependencies are installed from a pinned requirements file, application code is copied in, and the container runs uvicorn on port 7860. Isolating the dependency install layer from the application-code copy layer allows Docker's build cache to skip reinstalling dependencies when only source code changes.

6. Engineering Case Study: Integration and Deployment Failures

Bringing the system from a locally-passing implementation to a working deployed service surfaced a sequence of failures, several of which were caused by changes on third-party providers' side rather than defects in the application logic itself. This section documents each issue as encountered, the diagnostic approach used, and the resolution, since this process constituted a substantial part of the project's practical learning.

6.1 Silent exception handling

The chat endpoint wrapped the model call in a broad try/except that always returned HTTP 200 with a generic error string, discarding the underlying exception. This meant every distinct failure mode (bad credentials, missing model, provider outage) was indistinguishable from the client. The fix logged the full server-side traceback and surfaced the exception string to the frontend, which converted an unreproducible symptom into an immediately diagnosable one.

6.2 Invalid CORS configuration

The middleware combined `allow_origins=["*"]` with `allow_credentials=True`, a combination that is invalid under the CORS specification once credentials are involved. The endpoint did not use cookie-based auth, so the fix was to disable credentialed CORS entirely rather than enumerate explicit origins.

6.3 Frontend/backend port mismatch

The frontend's fetch target and the container's exposed port diverged after the deployment target changed, producing a generic network-level failure at the client with no server-side log entry at all — a useful diagnostic signal in itself, since a request that never reaches the server's logs points at the network path rather than the application.

6.4 API key format migration

Newly issued Google AI Studio credentials during the development period used a differently structured “auth key” format from the previously standard key prefix. The new-format key was, in this case, valid and correctly authenticated; the initial suspicion that the key was malformed was incorrect and was corrected once the current key format was confirmed against current provider documentation.

6.5 Model deprecation for new accounts

The originally selected model identifier began returning an HTTP 404 indicating it was no longer available to newly created accounts, ahead of its scheduled full retirement. The resolution was to reference the provider's floating “latest” alias for the model family rather than a dated model identifier, reducing the likelihood of the same failure recurring on a future deprecation cycle.

6.6 Unpinned SDK dependency risk

The Python client SDK for the model provider was unpinned in the dependency file. The provider's own package documentation warned that an upcoming major version would change automatic function-calling behavior, which the tool-augmented design in Section 3.2 directly depends on. The dependency was pinned below that major version boundary as a preventive measure.

Two observations generalize across these failures. First, several were invisible until the client's silent error handling (Section 6.1) was fixed, which underscores surfacing real error detail during development as a

prerequisite for diagnosing everything downstream of it. Second, multiple issues (6.4, 6.5, 6.6) originated from provider-side changes to authentication formats, model availability, and SDK behavior that occurred independently of any code change on the project's part, illustrating that a deployed integration against a third-party API carries ongoing maintenance exposure beyond the initial implementation.

7. Deployment and Hosting Strategy

7.1 API-based hosting evaluation

Several no-cost hosting targets were evaluated for the FastAPI backend. Hugging Face Spaces, the platform the project's Dockerfile was originally configured for (via its conventional port 7860), was found to have restricted free Docker-based Space creation to paid personal accounts during the development period, making it unsuitable as a genuinely free option at the time of evaluation. Render.com's free web service tier was selected instead: it deploys directly from a Dockerfile, requires no payment card, and imposes no fixed cost, at the trade-off of the service spinning down after a period of inactivity and incurring a short cold-start delay on the next request — an acceptable trade-off for a low-traffic personal portfolio site.

7.2 Migration to a self-hosted local model

As a subsequent iteration, the project explored replacing the hosted Gemini API with a locally-run model served through Ollama, removing dependency on a third-party inference provider and its account-level policy changes documented in Section 6. Running a local model continuously, however, has categorically different resource requirements from a lightweight API-calling backend: it requires persistent memory and CPU to hold the model in memory, which exceeds what typical free web-service tiers (including Render's free tier) provision.

7.3 Free-tier infrastructure for self-hosted inference

Oracle Cloud Infrastructure's Always Free tier was identified as the most viable no-cost option for continuously running a self-hosted model, since it provisions an actual persistent virtual machine rather than a request-scoped or time-limited free trial. The ARM-based Ampere A1 compute shape available under this tier was reduced by the provider partway through 2026 from 4 OCPUs / 24 GB RAM to 2 OCPUs / 12 GB RAM, which is a material constraint on which model sizes are practical to run; a quantized small model (in the 2–3 billion parameter range) fits comfortably within this allocation, while a 7-billion-parameter model consumes a much larger share of available memory and leaves little headroom for the API process itself. Provisioning free-tier ARM capacity is also subject to regional availability constraints on the provider's side, independent of account configuration.

8. Version Control Practices

Standard Git operations were used throughout development, including a case where two already-pushed commits needed to be discarded from the main branch. This required a hard reset of the local branch to the target commit followed by a force push, since the local and remote histories had diverged in a way a fast-forward push cannot resolve. force-with-lease was preferred over an unconditional force push, as it aborts the operation if the remote branch was updated by anything else since the last fetch, preventing an unintended overwrite of unseen remote work.

9. Conclusion and Future Work

The system as implemented demonstrates a working tool-augmented conversational agent grounded in a structured personal data source, deployed behind a decoupled frontend/backend architecture, with a documented path to both hosted-API and self-hosted-model operation under no-cost infrastructure. This report intentionally does not present quantitative performance figures (response latency, model output quality, or cost-per-request under load), as no formal measurement was conducted during development; such benchmarking, along with a basic per-IP rate limit on the chat endpoint to bound free-tier API quota consumption, is identified as concrete future work rather than reported as a completed result.

References

FastAPI documentation. FastAPI framework, high performance, easy to learn, fast to code.

Google AI for Developers. Gemini API documentation and the google-genai Python SDK reference.

Ollama documentation. Running and serving large language models locally.

Render documentation. Free tier web service deployment from a Dockerfile.

Oracle Cloud Infrastructure documentation. Always Free tier resource limits, Ampere A1 (Arm) compute shapes.

Docker documentation. Dockerfile reference and image build caching.

Git documentation. git-reset, git-push, and the --force-with-lease option.